

Deadlock

- A parallel program is **deadlocked** if it has not finished the execution and there's no possible evaluation step to take.
 - In example 3.c of Lecture 25 we can see that **await F then S end** can block forever, thus if this await statement is in one thread then this parallel program is deadlocked.
 - Deadlock is a runtime error; so, if a parallel program is deadlocked, the execution terminates (unsuccessfully) in pseudo-state $\perp_e$.
- 1. When are the following programs deadlocked?
 - a. **[await $x \geq 0$ then S_1 end || await $x \geq 0$ then S_2 end]**
If we execute it in a state σ with $\sigma(x) < 0$.
 - b. **[await $y \neq 0$ then $x := 1$ end || await $x \neq 0$ then $y := 1$ end]**
If we execute it in a state σ with $\sigma(x) = 0$ and $\sigma(y) = 0$.
 - c. $x := 1; y := 1; [\text{wait } y \neq 0; x := 0 \text{ || wait } x \neq 0; y := 0]$
Remember that **wait $y \neq 0 \equiv \text{await } y \neq 0 \text{ then skip end}$** . If we execute $x := 0$ before the test $x \neq 0$ in thread 2, then thread 2 will be blocked and the program is deadlocked; similarly, if we execute $y := 0$ before the test $y \neq 0$ in thread 1, then thread 1 will be blocked and the program is deadlocked.

This example shows that a longer atomic region is not always better: in this example, if we do not allow threads to interleave, there will be deadlocks.

- It is obvious that we want to know whether a program is deadlock-free or not. Like the test for disjointness program, for disjoint conditions and for interference freedom, we also have a “static and too strict” test for “deadlock freedom”.
- For threads (written in proof outlines) in the parallel composition $[\{p_1\} S_1^* \{q_1\} \parallel \{p_2\} S_2^* \{q_2\} \parallel \dots \parallel \{p_n\} S_n^* \{q_n\}]$ we define one of its **potential deadlock conditions** as $r_1 \wedge r_2 \wedge \dots \wedge r_n$, where each r_k is one of the follows:
 - $p \wedge \neg B$, if $\{p\} \text{ await } B \dots$ appears in S_k^*
 - q_k
 In addition, at least one r_k must be $p \wedge \neg B$. If $r_1 \wedge r_2 \wedge \dots \wedge r_n \equiv q_1 \wedge q_2 \wedge \dots \wedge q_n$, then we didn't look at any possible blocks for an await statements, so we don't consider $q_1 \wedge q_2 \wedge \dots \wedge q_n$ as a potential deadlock condition.
- The proof outline of program is **deadlock-free** if every one of its potential deadlock conditions is unsatisfiable; in other words, every one of its potential deadlock conditions is a contradiction.
- With the definition of deadlock-free, we can expand the domain of the parallelism rule to programs with await statements now.

Parallelism Rule (with await statements):

If for $k = 1 \dots n$, we have $\{p_k\} S_k^* \{q_k\}$ are all **interference-free and deadlock-free** proof outlines:

- 1 $\{p_1\} S_1^* \{q_1\}$
- 2 $\{p_2\} S_2^* \{q_2\}$

...
 $n \{p_n\} S_n^* \{q_n\}$
 $n+1 \{p_1 \wedge p_2 \wedge \dots \wedge p_n\} [S_1^* \parallel S_2^* \parallel \dots \parallel S_n^*] \{q_1 \wedge q_2 \wedge \dots \wedge q_n\}$ parallelism 1,2, ..., n

2. The following program $\{T\}[\mathbf{await} \ y \neq 0 \ \mathbf{then} \ x := 1 \ \mathbf{end} \parallel \mathbf{await} \ x \neq 0 \ \mathbf{then} \ y := 1 \ \mathbf{end}]\{\dots\}$ is from example 1.b. We use the following full proof outlines (using the await rule and forward assignment) for its two threads:

$\{T\} \mathbf{await} \ y \neq 0 \ \mathbf{then} \ \{y \neq 0\} \ x := 1 \ \{x = 1 \wedge y \neq 0\} \ \mathbf{end} \ \{x = 1 \wedge y \neq 0\}$
 $\{T\} \mathbf{await} \ x \neq 0 \ \mathbf{then} \ \{x \neq 0\} \ y := 1 \ \{x \neq 0 \wedge y = 1\} \ \mathbf{end} \ \{x \neq 0 \wedge y = 1\}$

Is this (proof outline of) program deadlock-free?

- We need to test the following potential deadlock conditions:

- $y = 0 \wedge (x \neq 0 \wedge y = 1)$ # r_1 is " $p \wedge \neg B$ ", r_2 is " q_2 " ✓
- $(x = 1 \wedge y \neq 0) \wedge x = 0$ # r_2 is " $p \wedge \neg B$ ", r_1 is " q_1 " ✓
- $y = 0 \wedge x = 0$ # r_1 is " $p \wedge \neg B$ ", r_2 is " $p \wedge \neg B$ " ✗
- (Note that, we don't test $q_1 \wedge q_2$)

Among the above three conditions, the first two are contradictions and the last one is not. So, this (proof outline of) program is not deadlock-free.

- a. What if we only run the above program only when $x \neq 0 \vee y \neq 0$?

In this case, the precondition for both threads become $p \equiv x \neq 0 \vee y \neq 0$:

$\{p\} \mathbf{await} \ y \neq 0 \ \mathbf{then} \ \{p \wedge y \neq 0\} \ x := 1 \ \{x = 1 \wedge y \neq 0\} \ \mathbf{end} \ \{x = 1 \wedge y \neq 0\}$
 $\{p\} \mathbf{await} \ x \neq 0 \ \mathbf{then} \ \{p \wedge x \neq 0\} \ y := 1 \ \{x \neq 0 \wedge y = 1\} \ \mathbf{end} \ \{x \neq 0 \wedge y = 1\}$

And the proof outline of this program becomes:

$\{p\} [\{p\} \mathbf{await} \ y \neq 0 \ \mathbf{then} \ \{p \wedge y \neq 0\} \ x := 1 \ \{x = 1 \wedge y \neq 0\} \ \mathbf{end} \ \{x = 1 \wedge y \neq 0\}$
 $\parallel \{p\} \mathbf{await} \ x \neq 0 \ \mathbf{then} \ \{p \wedge x \neq 0\} \ y := 1 \ \{x \neq 0 \wedge y = 1\} \ \mathbf{end} \ \{x \neq 0 \wedge y = 1\}] \{\dots\}$

- We need to test the following potential deadlock conditions:

- $(p \wedge y = 0) \wedge (x \neq 0 \wedge y = 1)$ # r_1 is " $p \wedge \neg B$ ", r_2 is " q_2 " ✓
- $(x = 1 \wedge y \neq 0) \wedge (p \wedge x = 0)$ # r_2 is " $p \wedge \neg B$ ", r_1 is " q_1 " ✓
- $(p \wedge y = 0) \wedge (p \wedge x = 0)$ # r_1 is " $p \wedge \neg B$ ", r_2 is " $p \wedge \neg B$ " ✓
- # $(p \wedge y = 0) \wedge (p \wedge x = 0) \Leftrightarrow (x \neq 0 \vee y \neq 0) \wedge (x = 0 \wedge y = 0)$
- $\Leftrightarrow (x \neq 0 \wedge x = 0 \wedge y = 0) \vee (y \neq 0 \wedge x = 0 \wedge y = 0)$
- $\Leftrightarrow F$

All three conditions are contradictions; thus, (the proof outline of) this program is deadlock free.

- From example 2, we have the following observations.
 - Even if some potential deadlock conditions are not contradiction, we still might not have deadlock at the runtime: $\{T\}[\mathbf{await} \ y \neq 0 \ \mathbf{then} \ x := 1 \ \mathbf{end} \parallel \mathbf{await} \ x \neq 0 \ \mathbf{then} \ y := 1 \ \mathbf{end}]\{\dots\}$ will be deadlocked only when $x = 0 \wedge y = 0$.
 - Strengthening the precondition can help us to avoid deadlocks: since in the above program deadlock only happens when $x = 0 \wedge y = 0$; by strengthening the precondition from T to $x \neq 0 \vee y \neq 0$ (this is like we add the domain predicate of the program to the precondition to avoid runtime error) we can prove a program is deadlock free.

3. Consider the following partial proof outline:

$$\begin{aligned} & \{ \dots \} [\{ p_1 \} \textbf{await } B_1 \textbf{ then } S_1 \textbf{ end } \{ q_1 \} \\ & \quad \| \{ p_{21} \} \textbf{await } B_{21} \textbf{ then } S_{21} \textbf{ end}; \{ p_{22} \} \textbf{await } B_{22} \textbf{ then } S_{22} \textbf{ end } \{ q_2 \} \\ & \quad \| \{ p_3 \} < S_3 > \{ q_3 \}] \{ \dots \} \end{aligned}$$

What are the potential deadlock conditions we need to test?

○ Let's denote D_1, D_2 and D_3 as the set of all possible expressions for r_1, r_2 and r_3 respectively. Then,

$$D_1 = \{ p_1 \wedge \neg B_1, q_1 \}$$

$$D_2 = \{ p_{21} \wedge \neg B_{21}, p_{22} \wedge \neg B_{22}, q_2 \}$$

$$D_3 = \{ q_3 \}$$

○ Thus, there are $2 * 3 * 1 - 1 = 5$ different potential deadlock conditions:

- $(p_1 \wedge \neg B_1) \wedge (p_{21} \wedge \neg B_{21}) \wedge q_3$
- $(p_1 \wedge \neg B_1) \wedge (p_{22} \wedge \neg B_{22}) \wedge q_3$
- $(p_1 \wedge \neg B_1) \wedge q_2 \wedge q_3$
- $q_1 \wedge (p_{21} \wedge \neg B_{21}) \wedge q_3$
- $q_1 \wedge (p_{22} \wedge \neg B_{22}) \wedge q_3$